

WAF, Reverse Proxies & Content Filters

The WAF job

A Web Application Firewall filters HTTP(S) *before* the app sees it: blocks SQLi/XSS payloads (signature + behavior), geo/rate rules, virtual patching.

ModSecurity / OWASP CRS (the open stack)

```
# apt install libapache2-mod-security2 (or nginx+rules)
# wire OWASP CRS into modsec:
SecRule REQUEST_URI "@contains /cgi-bin/" "deny"
# Great examples:
SecRule REQUEST_BODY "@rbl research.example" "phase:2,deny"
```

CRS gives you out-of-the-box 2000+ rules with site-wide tuning (paranoia levels 1-4).

Virtual patching

When an app bug can't be fixed fast, a WAF rule blocks the exploit path until the real patch lands. Essential practice for unpatcheable legacy.

Reverse proxy duties

```
Client --> [Proxy: TLS terminate, cache, header normalizing, limits] --> App
```

TLS termination & HSTS (so apps can be http internally)

Header safety (strip client IP spoofing; enforce Host)

Caching; CDN-like offload

Path filtering: block /admin from external, allow internal routes

WAF limits (honest)

It's *before* code: logic flaws (A04) and business logic are invisible
Bypass arms race: encodings, chunk tricks, semantic equivalence
Never treat WAF as 'the fix' – it's a delay/cover while you patch

Config craft

Run in detection/log-only → tune → then enforce
Watch false positives (blocking legit users) – the #1 WAF killer
Multiple modes: blocklist (block known-bad) or allowlist (only-what-app-does)

Deploy a WAF like a shield: it deflects what you know, buys time for what you don't, and never replaces armor (secure code).

[← IDS/IPS: Suricata, Rules, De...](#)

[Segmentation, VPN & ZTNA →](#)